

Paiement multi-fournisseurs switch & parallèle

Décision d'architecture : faire cohabiter Stripe et les PSP régionaux (PayZone, CMI, PayTabs) en parallèle, switchables par org / pays / devise / flux.

Statut	Accepté · 13 juillet 2026
Contexte	Lancement Maroc — Stripe absent → PSP locaux en parallèle
Constat	La fondation existe déjà — cet ADR la complète et la généralise

ADR — Architecture de paiement multi-fournisseurs

- **Statut** : Accepté (2026-07-13)
- **Décideurs** : Toufik (tech) · fondateurs (Jamil Mazy, Khaoula El Haouije, Amro Sunbul)
- **Contexte marché** : lancement **Maroc d'abord** → Stripe **n'opère pas au Maroc**. Besoin de faire cohabiter Stripe (international) et des PSP régionaux (PayZone/CMI au Maroc, PayTabs en KSA) **en parallèle** et de **switcher** selon l'org / le pays / la devise / le flux.

1. Contexte

Le produit doit encaisser et reverser de l'argent sur **plusieurs juridictions** avec des fournisseurs différents, **en parallèle** (une org peut accepter EUR via Stripe **et** MAD via un PSP local) et **switchable** sans réécrire les flux métier.

Constat de l'audit du code (2026-07-13) : la fondation existe déjà et elle est bien conçue. Cet ADR ne crée pas l'architecture — il la **formalise, la complète et la généralise**.

Ce qui existe

Deux ports (pattern Strategy + registry, façon ports & adaptateurs) :

Port	Interface	Adaptateurs	Rôle
Entrant	PaymentProvider	Stripe , PayZone (MA,réal), PayTabs , CMI ,(PayPal)	createPayment / capture / refund / customer / webhook
Sortant	PayoutExecutor	StripeConnect , Wise , SEPA , OpenBanking , Manual	Exécution des payouts, idempotente

- **Orchestrateur-resolver** : PaymentOrchestrationService — résout le provider par preferredProvider → **devise** (MAD → CMI puis PayZone , SAR → PayTabs) → **pays** → fallback Stripe. Idempotence, ledger PaymentTransaction , outbox Kafka, circuit breaker.
- **Config par organisation** : PaymentMethodConfig (enabled, sandbox, clés **chiffrées**) via PaymentMethodConfigService . **Plusieurs providers actifs en parallèle** par org, la devise de la transaction tranche.
- Le ManualPayoutExecutor couvre déjà le « **versé hors plateforme, tracé** » requis pour le Maroc au lancement.

2. Problème (les gaps)

1. **Adoption incohérente — gap n°1**. L'orchestrateur est le **seul** consommateur du registry, mais **41 fichiers appellent Stripe en direct** (booking checkout, cautions, abonnement, payout ménage, mobile, shop, upsells, crédits IA...). Le « switch/parallèle » ne vaut **que** pour les flux passant par l'orchestration.
2. **Flux non couverts par le port entrant** : **capture/caution** (pré-autorisation, SetupIntent), **abonnement récurrent** (Stripe Billing en direct), **checkout sessions**.

3. **Pas de modèle de capacités** : les providers lèvent `UnsupportedOperationException` (PayZone → pas de payout) — entorse ISP ; le resolver peut choisir un provider incapable du flux.
4. **Adaptateurs régionaux non certifiés** : PayZone/CMI/PayTabs codés mais champs API « à confirmer à l'onboarding » — non testés en sandbox réel.
5. **⚠ Entorse money-safety** : `PaymentOrchestrationService` est `@Transactional` (niveau classe) et appelle `provider.createPayment()` (**HTTP externe**) dans la transaction → viole la règle « jamais d'appel HTTP externe dans une transaction DB » (CLAUDE.md).

3. Décision

3.1 Décisions de conception (tranchées)

#	Décision	Choix retenu
D1	Abonnement SaaS récurrent	Port dédié <code>SubscriptionProvider</code> (Stripe Billing / PayZone récurrent), séparé du paiement one-shot — la sémantique récurrente diffère trop du one-shot.
D2	Capacités des providers	Capacités déclarées (<code>PAY</code> / <code>CAPTURE</code> / <code>REFUND</code> / <code>PAYOUT</code> / <code>RECURRING</code> / <code>CARD_ON_FILE</code>) + resolver capability-aware ; suppression des <code>UnsupportedOperationException</code> .
D3	Caution au Maroc	Capability-gated : pré-autorisation via Stripe uniquement ; au lancement MA, caution manuelle / empreinte (pas de pré-auth PSP local).
D4	Ordre de migration	Strangler, non-breaking , par vagues (§5).

3.2 Architecture cible

Tous les flux métier passent par l'orchestration ; **plus aucun appel Stripe direct** hors des adaptateurs.

ARCHITECTURE CIBLE

Flux métier · booking · caution · abonnement · ménage · shop · IA · mobile

↓ aucun appel Stripe direct — uniquement l'orchestration

PaymentOrchestrator

résout : preferred → capacité → devise → pays → fallback · ledger `PaymentTransaction` · idempotence · webhooks unifiés · transactions courtes (appel externe hors tx)

Port entrant

PaymentProvider + capacités
Stripe · PayZone · CMI · PayTabs · PayPal

Port abonnement

SubscriptionProvider + capacités
Stripe Billing · PayZone récurrent

Port sortant

PayoutExecutor + capacités
Stripe Connect · Wise · SEPA · OpenBanking · Manual

Config par organisation (enabled / sandbox / clés chiffrées) — providers actifs **EN PARALLÈLE**

Composants à ajouter/compléter :

1. `PaymentCapability` (enum) + déclaration par adaptateur + resolver capability-aware.
2. `SubscriptionProvider` (port abonnement) : `Stripe Billing` + `PayZone récurrent` (MA).

3. Extension de l'orchestration à la **capture/caution** (capability-gated).
4. **Migration strangler** des 41 sites Stripe directs derrière l'orchestration.
5. **Correction de l'entorse transactionnelle** (tx courtes, appel externe hors tx, effets post-commit).
6. **Certification sandbox** PayZone/CMI/PayTabs.

4. Invariants money-safety (à préserver — CLAUDE.md)

- `StripeGateway` : `RequestOptions` par appel + idempotency keys (jamais `Stripe.apiKey` global). S'applique par analogie à chaque adaptateur régional (creds par org, clé d'idempotence).
- `StripeAmounts.toMinorUnits` (`HALF_UP`) pour euros→centimes ; équivalent par devise pour les PSP régionaux (attention aux devises **sans sous-unité** ou à 3 décimales).
- `PaymentStatusTransitionService` : transitions de statut en **UPDATE conditionnel (CAS)** — anti double-crédit sur re-livraison webhook.
- **Jamais d'appel HTTP externe dans une transaction DB** : tx courte (persist `PENDING`) → appel provider **hors tx** (idempotent) → nouvelle tx (persist résultat) ; effets externes post-commit via `afterCommit`.
- **Le serveur recalcule toujours le montant** depuis l'entité métier — le montant client n'est qu'un cross-check.
- **Pas de** `catch(Exception)` **avaleur** : échec → statut de réconciliation explicite + notification admin.

5. Plan de migration (strangler, non-breaking)

Vague	Contenu	Sortie attendue
V1 — Fondations	<code>PaymentCapability</code> + resolver capability-aware + fix entorse transactionnelle	Socle prêt, zéro régression (tests de caractérisation d'abord)
V2 — Encaissement voyageur	Booking checkout, cautions, balance/deferred → via orchestration	Paiements voyageurs multi-provider (Stripe + PSP)
V3 — Abonnement SaaS	<code>SubscriptionProvider</code> (Stripe Billing + PayZone récurrent MA)	Facturation self-serve multi-pays
V4 — Payouts	<code>HousekeeperPayout</code> + <code>OwnerPayout</code> → via <code>PayoutExecutor</code> (dont <code>Manual</code> MA)	Payout ménage/proprio multi-rail, MA en manuel tracé
V5 — Périphérie	Shop, upsells, crédits IA, mobile → via orchestration	Plus aucun Stripe direct hors adaptateurs
V6 — Certification	Sandbox PayZone/CMI/PayTabs + E2E multi-devise	Adaptateurs régionaux prouvés

Méthode par vague : tests de caractérisation sur le flux avant migration → bascule derrière l'orchestration → vérif E2E → suppression du code Stripe direct devenu mort (preuve avant suppression).

6. Conséquences

Positives : un seul point d'encaissement/reversement ; Maroc débloqué (PayZone + Manual) sans réécrire les flux ; parallèle multi-devise par org ; testabilité (adaptateurs mockables) ; conformité money-safety renforcée (entorse corrigée).

Négatives / coûts : la migration strangler touche ~41 fichiers (étalée, non-breaking) ; certification des PSP régionaux dépend d'un onboarding réel (délai externe) ; sémantique caution dégradée au Maroc au lancement (manuel).

7. Décisions encore ouvertes (hors périmètre de cet ADR)

- Entité juridique d'encaissement (FR/US via Stripe Atlas) pour la part **EUR/USD** internationale.
- Prix exact de l'abonnement (per-listing + add-ons) — indépendant de l'architecture.
- Priorité exacte V2 vs V3 selon la date de lancement visée.